Task 1: Guided Activity, User profile with missing data:

```
import Foundation
struct Address {
   var city: String?
}

struct User {
   var name: String
   var age: Int?
   var address: Address?
}

func describe(_ user: User) -> String {
   guard let age = user.age else {
      return "\(user.name): Age is unknown"
   }

   let city = user.address?.city ?? "unknown city"

   return "Name: \(user.name), Age: \(age), City: \(city)"
}

let user1 = User(
   name: "Asha",
   age: 21,
   address: Address(city: "Chennai")
)

let user2 = User(
   name: "Ravi",
   age: nil,
```

```
    address: Address(city: "Coimbatore")
)

let user3 = User(
    name: "Priya",
    age: 20,
    address: Address(city: nil)
)

print(describe(user1))
print(describe(user2))
print(describe(user3))

let number1 = Int("42")
let number2 = Int("abc")

print(number1 as Any)
print(number2 as Any)
```

output:

Output

```
Name: Asha, Age: 21, City: Chennai
Ravi: Age is unknown
Name: Priya, Age: 20, City: unknown city
Optional(42)
nil
```

Task 2: Practice Task, Safe number parsing:

```
import Foundation

let inputs = ["10", "x", "25", "", "7.5", "40"]

let validNumbers = inputs.compactMap { Int($0) }
```

```
print("Valid Integers:", validNumbers)



let sum = validNumbers.reduce(0, +)

print("Sum:", sum)


let invalidCount = inputs.count - validNumbers.count

print("Invalid Entries:", invalidCount)


let doubled = Int(inputs.first.flatMap { Int($0) }.map { $0 * 2 } ?? 0)

print("First Valid Number Doubled:", doubled)
```

output:

Output

```
Valid Integers: [10, 25, 40]
Sum: 75
Invalid Entries: 3
First Valid Number Doubled: 20
```

Task 3: Debug Task, Remove the crashes:

```
import Foundation

let scores = ["Asha": 90, "Ravi": 72]


if let kiran = scores["Kiran"] {
   if let number = Int("12a") {
      print(kiran + number)
   } else {
      print("Error: The string cannot be converted to an integer.")
   }
} else {
   print("Error: Kiran's score was not found.")
}
```

Output:

Output

```
ERROR!
Error: Kiran's score was not found.
```